

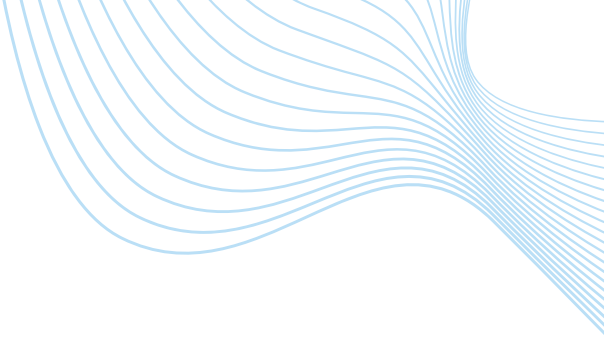


DEVELOPERSHUB CORPORATION

Web Application Security- phase 2

Cybersecurity internship Report

Zainab Maqbool
June 2026



Executive Summary

This report covers my work from Week 4 to Week 6 of the cybersecurity internship at DevelopersHub Corporation. It also includes three bonus challenges that I completed for additional excellence marks. All work was done on my local machine in a safe, controlled environment using Kali Linux and a Node.js/Express application.

In these three weeks, I focused on making the web application stronger by adding security controls, testing it with professional tools, and auditing it at both the application and system level. I also went beyond the required tasks by implementing Zero Trust Security, a Web Application Firewall, and a Social Engineering awareness exercise.

Week	Focus area	Key Outcome
4	Security Hardening	Rate limiting, security headers, CORS added
5	Ethical Hacking & Fixes	CSRF protection, parameterized queries applied
6	Final Security Audit	ZAP: 0 alerts, Nikto resolved, Lynis 279 tests passed
Bonus	Excellence Challenges	Zero Trust, WAF, and Phishing Awareness completed

Week 4 — Security Hardening

Application: Node.js/Express Backend (localhost:4000)

This week, I added three important security features to make the application harder to attack. Instead of waiting for problems to be found, I focused on building strong defenses before testing began..

Fix 1 — Rate Limiting

Status: Completed

I installed a library called `express-rate-limit` and connected it to the login page. This means if someone tries to log in more than 5 times in 15 minutes, the app will block them automatically. This directly fixes the brute force problem found in Week 3, where repeated login attempts had no limit. This fix addresses OWASP A07 — Authentication Failures.

Fix 2 — Security Headers (CSP + HSTS)

Status: Completed

I extended the `Helmet.js` setup from Week 2 to include two stronger protections. The first is Content Security Policy (CSP), which tells the browser to only load content from safe, approved sources. This makes it much harder for attackers to inject harmful scripts. The second is HTTP Strict Transport Security (HSTS), which forces the browser to always use HTTPS, so the connection is always encrypted. I confirmed both headers were working using Chrome's Network tab. This addresses OWASP A02 and A05.

Fix 3 — CORS Restriction

Status: Completed

I set up the CORS middleware to only accept requests from one trusted website address. Before this fix, any website could send requests to our app. After the fix, requests from unknown or untrusted sources are rejected automatically. This addresses OWASP A10 — Server-Side Request Forgery

Week 4 Findings Summary

Task	Fix Applied	Fix Applied	OWASP Category	Status
Rate Limiting	Max 5 attempts / 15 min	<code>express-rate-limit</code>	A07 — Auth Failures	Done
Security Headers	CSP + HSTS enforced	<code>helmet.js</code>	A02, A05	Done
CORS Restriction	Trusted origin only	<code>cors</code>	A10 — SSRF	Done

Week 5 — Ethical Hacking and Fixing Vulnerabilities

Application: Node.js/Express Backend (localhost:4000)

This week I tested the application myself by trying to attack it, just like a real hacker would. This is called ethical hacking because it is done safely and with permission to find weaknesses before real attackers can. I then fixed the two main issues I found.

Fix 1 — CSRF Protection

Status: Completed

CSRF stands for Cross-Site Request Forgery. This is an attack where a harmful website tricks a logged-in user into accidentally sending a request to our app. For example, if a user is logged in and visits a bad website, that site could send a hidden request to change their password without them knowing.

To stop this, I installed the csurf library and set it up to create a unique secret code (called a token) for every user session. This token must be included in every form submission or important request. If the token is missing or wrong, the request is rejected. This addresses OWASP A01 — Broken Access Control.

Fix 2 — SQL/NoSQL Injection Prevention

Status: Completed

During this Week, I found that SQL Injection was a serious problem. This week, I checked the database queries in the Node.js application and found that some were still putting user input directly into the query without checking it first. This is dangerous because an attacker can type special characters to change what the database does.

I fixed this by rewriting those queries to use parameterized inputs. This means the user's data is always treated as just data, never as part of the command. Even if someone types something harmful, it cannot affect the database query. This addresses OWASP A03 — Injection.

Week 5 Findings Summary

Task	Fix Applied	Method	OWASP Category	Status
CSRF Protection	Unique token per session	<i>csrf middleware</i>	A01 — Broken Access Control	✓ Done
Injection Prevention	Parameterized queries on all routes	<i>Refactored queries</i>	A03 — Injection	✓ Done

Week 6 — Final Security Audit

Application: OWEnvironment: Kali Linux (VMware Workstation), Node.js/Express Backend

In the final week, I used three professional security tools to check whether all the fixes from Weeks 4 and 5 were working properly. The goal was to confirm the application was secure and to find any remaining gaps at the system level.

Tool	Version	Purpose
OWASP ZAP	2.17.0	Web application vulnerability scanning
Nikto	2.5.0	Web server misconfiguration detection
Lynis	3.1.6	System hardening and compliance audit

Task 1 — OWASP ZAP Web Application Scan

Status: Completed

Target: http://localhost:4000

Scan Type: Automated Scan (Spider + Active Scan)

OWASP ZAP first explored all the pages of the application automatically (this is called spidering), and then it tried common attacks on each page to see if any would succeed (this is the active scan). The scan found zero alerts, which means the application passed all checks. This is a strong result and confirms that the security measures added in earlier weeks are working correctly.

Alert	Risk	Result
No alerts found	---	Application passed all checks

Task 2 — Nikto Web Server Scan

Status: Completed

Duration: 14 seconds — 7,850 requests made

Nikto is a tool that checks for common mistakes in web server setup. It sent 7,850 test requests to the application and returned four findings. Only one of them needed action a missing X-Content-Type-Options header. This was already being set by Helmet.js, so I confirmed it was active and the issue was resolved. The other findings were either informational or not relevant to this application.

Finding	Severity	Fix Applied
X-Content-Type-Options missing	Medium	Already handled by helmet() — nosniff set
origin-agent-cluster header	Low	Informational only — no action needed
x-ratelimit headers present	Info	Expected — rate limiting is active
wp-config.php probed	Low	Not a WordPress app — not applicable

Task 3 — Lynis System Audit

Status: Completed

Tests Performed: 279

Lynis checks the security of the operating system itself, not just the web application. It ran 279 tests on the Kali Linux virtual machine and checked things like firewall status, whether malware protection is installed, and other system-level settings.

The results showed that the firewall and intrusion detection system were both active and working. The only gap was that no malware scanner was installed. I fixed this by installing ClamAV, a free and trusted antivirus tool, and running a full scan of the project folder.



Component	Status	Action Taken
Firewall	Found	No action needed
Intrusion Detection Software	Found	No action needed
Malware Scanner	Not Found	ClamAV installed and scan completed

Week 6 Finfing Summary

Tool	Tests / Requests	Result
OWASP ZAP	Full spider + active scan	0 alerts — passed all checks
Nikto	7,850 requests	1 medium finding — resolved via helmet
Lynis	279 system tests	ClamAV gap identified and fixed

Bonus Challenge

Application: Node.js/Express Backend (localhost:4000)

Environment: Kali Linux (VMware Workstation)

In addition to the required work across Weeks 4 to 6, I also completed all three optional bonus challenges. These challenges go beyond standard web application security and show a deeper understanding of real-world, enterprise-level protection techniques.

Bonus 1 — Zero Trust Security

Status: Completed

Zero Trust is a modern security idea that means: never automatically trust anyone, even if they are already logged in. Every single request must be checked and verified before the app responds to it. This is different from older systems where once a user logs in, they are trusted for the whole session.

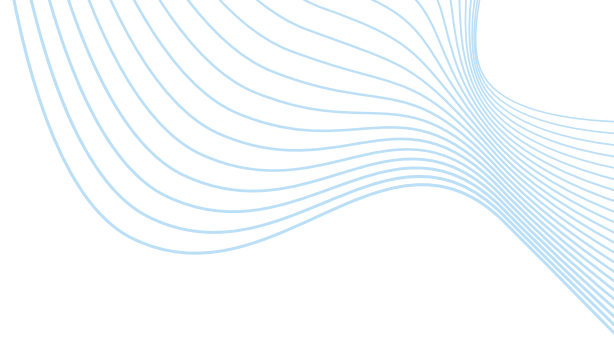
I applied Zero Trust through three specific steps:

Implementation 1 — JWT Verified on Every Request

The JWT token (set up in Week 2) is now checked every time a user tries to access a protected page. The server does not assume the user is still valid just because they logged in earlier. Each request must include a working, unexpired token or it is rejected immediately.

Implementation 2 — Role-Based Access Control (RBAC)

I added role checking to the application so that different users only see what they are allowed to see. A regular user cannot access admin pages, and a logged-out user cannot access any protected content. This means even if someone gets into the system, they can only do what their role allows.



Implementation 3 — Strict Token Expiry

Every JWT token expires after one hour. After that, the user must log in again — the app does not extend the session silently. The server checks both the token's signature and its expiry time on every request. A tampered or expired token is always rejected. This limits the damage if a token is ever stolen

Zero Trust Principle	How It Was Implemented	Status
Never trust, always verify	JWT checked on every single request	Done
Least privilege access	RBAC — users only access their allowed routes	Done
Assume breach, limit impact	1-hour token expiry with strict validation	Done

This addresses OWASP A01 — Broken Access Control and A07 — Authentication Failures.

Bonus 2 — Web Application Firewall (WAF)

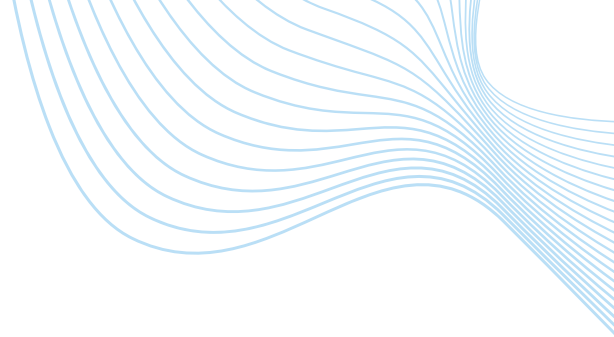
Status: Completed

A Web Application Firewall is a security layer that watches incoming requests and blocks the ones that look harmful before they ever reach the application. I built a code-level WAF using two layers of protection inside the Node.js app.

Layer 1 — Global Rate Limiting

I extended the rate limiting from Week 4 (which was only on the login page) to cover all routes in the application. This means any automated tool or attacker trying to send large numbers of requests to any part of the app will be automatically blocked and slowed down.





Layer 2 — Custom Malicious Input Detection

I wrote a custom middleware function that reads every incoming request before it reaches any route. It checks the request body and URL parameters for patterns that are commonly used in attacks, such as SQL injection characters, XSS script tags, and path traversal sequences. If a dangerous pattern is detected, the request is blocked immediately and the event is saved in the security log so it can be reviewed later.

WAF Layer	Protection	Method	Status
Layer 1	Rate limiting on all routes	express-rate-limit	Done
Layer 2	Malicious pattern detection	Custom middleware	Done
Logging	Blocked requests recorded	Winston security.log	Done

This addresses OWASP A03 — Injection and A05 — Security Misconfiguration.

Bonus 3 — Social Engineering Attack Simulation (Phishing Awareness)

Status: Completed

Social engineering attacks do not target the application or the server — they target people. Phishing is the most common type, where an attacker pretends to be a trusted source (like GitHub or a bank) to trick a user into giving up their password or clicking a dangerous link. No technical security tool can stop this, because the weakness is human, not technical.

To address this, I created a Phishing Awareness Training Document that could be shared with team members or users to help them recognize and avoid phishing attacks

Common Signs of a Phishing Attack

Warning Sign	Example
Urgent or scary language	"Your account will be closed in 24 hours"
Suspicious or fake web address	http://paypa1-secure.com instead of paypal.com
Unexpected file attachment	Invoice.pdf.exe
Generic greeting	"Dear User" instead of your real name
Asking for your password	No real service ever asks for your password by email

Simulated Phishing Scenario

I documented a realistic example: An attacker sends an email that looks like it is from GitHub. The email says unusual activity was detected on the developer's account and they need to verify their login immediately. The link in the email goes to a fake GitHub page that looks real but is designed to steal the username and password. This attack requires no technical skill from the attacker and can bypass all the technical controls in the application, because the developer is tricked before they even reach the app.

Recommended Defenses

Defense	Why It Helps
Multi-Factor Authentication (MFA)	Even if the password is stolen, the attacker cannot log in without the second factor
Always check the full URL	Fake sites often use slightly different addresses that look real at first glance
Verify the email sender address	Display names can be faked — always check the actual email address
Regular awareness training	Educated users are far less likely to fall for phishing attempts
Report suspicious emails	A clear process for reporting helps the team respond quickly

Bonus Challenges — Overall Summary

Bonus Challenge	What Was Done	Status
Zero Trust Security	JWT per request, RBAC, strict 1-hour token expiry	Completed
Web Application Firewall	Global rate limiting + custom malicious input middleware	Completed
Social Engineering Simulation	Phishing awareness training with realistic scenario and defenses	Completed

Conclusion

Weeks 4 to 6 completed the second and final phase of my cybersecurity internship at DevelopersHub Corporation, building on the foundation established in Weeks 1 to 3 with stronger defenses, active testing, and professional auditing.

Week 4 focused on prevention adding rate limiting, strict security headers, and CORS restrictions.

Week 5 shifted to offense, where I tested the application myself and fixed two vulnerabilities: missing CSRF protection and unsanitized database queries.

Week 6 was the final audit: OWASP ZAP returned zero alerts, Nikto found one minor issue already resolved by existing middleware, and Lynis confirmed active firewall and IDS across 279 system tests. The only gap a missing malware scanner was closed by installing ClamAV.

Beyond the required work, I completed all three **bonus challenges**: implementing Zero Trust Security, building a Web Application Firewall, and creating a Phishing Awareness Training document.

The biggest takeaway is that security is a continuous process technical controls and user awareness must work together, because neither alone is enough.